Key-Value Databases

1. Overview

Key-value databases store data as simple pairs:

```
None
key → value
```

Each unique **key** maps to a corresponding **value**.

Example:

```
None
user:101   → {"name":"Samarth"}

session:55 → token-data

cart:9001  → items-list
```

They are one of the fastest database models for simple lookups.

2. Core Idea

Instead of tables + joins, key-value stores use direct retrieval:

```
None
GET(key)

PUT(key,value)

DELETE(key)
```

This makes reads/writes extremely fast.

3. Why Performance Is Good

Keys are commonly indexed using:

- Hash tables
- Partition maps
- B-trees (some engines)
- In-memory indexes

Hashing allows fast lookup:

```
None
hash(key) → storage location
```

So reads are often near O(1) average lookup time.

4. Keys

Keys should be:

- Unique
- Deterministic
- Small enough
- Hashable / serializable
- Easy to partition

Examples:

```
None
user:101

session:abc123

cart:9001

product:501
```

5. Values

Values can be:

- String
- Number
- JSON
- Binary blob
- Serialized object
- List / set / hash (depends on system)

Example:

```JSON
{

  "name": "Samarth",

  "city": "Surat"

}
```

6. Popular Examples

- Redis
- Memcached
- Amazon DynamoDB
- Riak

7. Common Use Cases

Best for:

- Cache
- Sessions
- Shopping carts
- User tokens
- Feature flags
- Rate limiting counters
- Leaderboards
- Fast metadata lookup

8. Key-Value as Cache

Very common architecture:

```
None
Client Request
   ↓
Check Redis Cache
   ↓ hit
Return fast


   ↓ miss
Query DB
Store in cache
Return result
```

9. Why Cache Doesn’t Need Same Availability as DB

If cache fails:

```
None
Fallback to primary database
```

System may slow down but still works.

So cache availability needs are often lower than source-of-truth DB.

10. Eviction Policies

When memory full, remove items.

Common strategies:

LRU (Least Recently Used)

Remove least recently accessed key.

LFU (Least Frequently Used)

Remove least used.

TTL

Auto expire after time.

Example:

```
None
session:123 expires in 30 min
```

11. Redis vs Memcached

Feature	Redis	Memcached

Persistence	Yes (optional)	No
Data Types	Rich	Simple
Replication	Yes	Basic
Speed	Very fast	Very fast
Use Case	Cache + more	Pure cache

12. In-Memory Performance

Many key-value systems keep data in RAM.

Benefits:

- Microsecond/millisecond latency
- Very high throughput

Trade-off:

- RAM costlier than disk

13. Distributed Key-Value Stores

At scale, data partitioned across nodes.

```
None
hash(key) % N → node
```

This distributes load.

Used in:

- Redis Cluster
- DynamoDB
- Riak

14. Consistency Trade-offs

Distributed key-value stores may use:

- Strong consistency
- Eventual consistency
- Tunable consistency

Depends on product.

15. Example Operations

```
None
SET session:123 xyz

GET session:123

DEL session:123

INCR page_views
```

Counters are common.

16. Best Interview Use Cases

Say key-value store for:

- Login sessions
- OTP storage
- Rate limiter counters
- Product page cache
- Cart state
- Fast token validation

17. Limitations

No joins
Query by non-key difficult
Limited relational constraints
Complex analytics not ideal
Need careful key design

18. Key Design Best Practices

Use namespaced keys:

```
None
user:101

cart:101

order:5001
```

Benefits:

- Easier debugging
- Better organization

19. Interview Answer Example

Q: Why Redis here?

> I'd use Redis because we need ultra-fast reads/writes for sessions and cache, low latency, TTL support, and counters for rate limiting.

20. Short Revision Notes

```
Key-value DB:

key → value


Best for:

- Cache

- Sessions

- Counters

- Cart

- Fast lookups


Examples:

Redis, Memcached
```

21. One-Line Summary

Key-value databases store data as direct key-to-value mappings, providing extremely fast lookups and making them ideal for caching, sessions, counters, and other low-latency access patterns.